iWithYou — 问题修复计划书

> 创建日期：2026-03-27
>
> 关联文档：ARCHITECTURE.md | ARCHITECTURE_REVIEW.md

本计划书针对架构审查中发现的 **31 个问题**，按优先级排列为 **4 个阶段**、**31 个具体任务**，每个任务包含问题定位、修改文件、实现步骤和验收标准。

阶段总览

阶段	主题	任务数	目标周期
一	紧急安全修复	9	第 1 周
二	安全加固	10	第 2-3 周
三	工程规范化	8	第 4-8 周
四	架构演进	4	第 9-16 周

阶段一：紧急安全修复（第 1 周）

> 本阶段所有任务涉及正在暴露的安全漏洞，必须立即处理。

任务 1.1：轮换所有已泄露凭证

对应问题：C-1

优先级：🔴 P0 — 立即执行

背景

以下凭证已在 Git 历史或源码中暴露，无论仓库是否公开，都应视为已泄露：

操作清单

#	凭证	轮换平台	状态
1	MySQL 密码	数据库服务器	☐
2	JWT Secret	后端 .env	☐
3	微信 AppSecret	微信开放平台	☐
4	腾讯云 SecretKey	腾讯云控制台 → 访问管理 → API 密钥	☐
5	阿里云 OSS AccessKey	阿里云控制台 → RAM	☐
6	APNs 私钥	Apple Developer → Keys	☐
7	华为推送 AppSecret	AppGallery Connect	☐
8	荣耀推送 AppSecret	荣耀开发者中心	☐
9	小米推送 AppSecret	小米推送控制台	☐
10	OPPO 推送 AppSecret	OPPO 开放平台	☐
11	VIVO 推送 AppSecret	VIVO 开放平台	☐
12	友盟 AppMasterSecret	友盟控制台	☐
13	极光 MasterSecret	极光控制台	☐
14	DeepSeek API Key	DeepSeek 控制台	☐

验收标准

- [ ] 所有旧凭证已作废
- [ ] 新凭证已在生产环境中验证可用
- [ ] 服务重启后功能正常

任务 1.2：源码中移除所有硬编码密钥

对应问题：C-1

优先级：🔴 P0

涉及文件

文件	修改内容
`backend/run_New.sh`	移除所有内联凭证，改从 `.env` 文件加载
`frontend/lib/configure.dart`	移除 `cloudSecretId`、`cloudSecretKey`、UMeng AppKey
`frontend/lib/app/core/services/push/mypushconfig.dart`	移除所有 AppSecret，仅保留 AppId/AppKey（公开标识）

实现步骤

步骤 1：后端 `run_New.sh` 改造

将：

```
docker run -e MYSQL_PASSWORD=SZb808jq@755 -e JWTSECRET=xxx ...
```

改为：

```
# run_New.sh
set -a
source .env.production   # 从环境文件加载
set +a

docker run --env-file .env.production -d $repository
```

创建 `.env.production.example`（不含真实值，仅作模板）：

```
MYSQL_PASSWORD=<your-mysql-password>
JWTSECRET=<your-jwt-secret>
# ...其他变量
```

步骤 2：前端 `configure.dart` 改造

删除：

```
static const cloudSecretId = "";
static const cloudSecretKey = "";
```

替换为后端中转：

```
/// 云存储上传凭证改为从后端获取临时 STS 令牌
/// 参见：POST /api/upload/sts-token
static const uploadStsEndpoint = '/api/upload/sts-token';
```

步骤 3：前端 `mypushconfig.dart` 改造

保留（公开标识，安全）：

```
static const String huaweiAppId = '110807363';
static const String xiaomiAppId = '2882303761520275257';
```

删除所有 Secret：

```
// 删除以下所有行
// static const String huaweiAppSecret = '6c44c57...';
// static const String honorAppSecret = '625f37e...';
// static const String oppoAppSecret = '649aff7...';
// static const String xiaomiAppSecret = 'rJX9LE...';
// static const String vivoAppSecret = '407b76d...';
// static const String umengMessageSecret = '71e294...';
```

> 推送厂商的 Secret 仅在服务端使用，客户端无需也不应持有。

步骤 4：清除 Git 历史

```
# 安装 bfg-repo-cleaner
brew install bfg

# 备份仓库
cp -r iwithyou iwithyou-backup

# 清除历史中可能存在的 .env 文件
bfg --delete-files '.env' --delete-files '.env_remote'

# 清除历史中的密钥字符串
bfg --replace-text passwords.txt
# passwords.txt 内容:
# SZb808jq@755 ==> ***REMOVED***
# c6d5d17a-4ad2-4e2b-8475-c86e25531864 ==> ***REMOVED***
# （列出所有已知密钥值）

git reflog expire --expire=now --all && git gc --prune=now --aggressive
```

验收标准

- [ ] `git log --all -p | grep -i "SZb808jq"` 无结果
- [ ] `grep -r "cloudSecretKey" frontend/lib/` 无结果
- [ ] `grep -r "AppSecret" frontend/lib/` 无结果（除注释外）
- [ ] `.env.production` 在 `.gitignore` 中
- [ ] 服务使用新凭证正常运行

任务 1.3：修复 userId 注入漏洞

对应问题：C-2

优先级：🔴 P0

涉及文件

`backend/lib/routes/json_routes/api_routes.dart`

实现步骤

步骤 1：创建认证上下文辅助函数

在 `api_routes.dart` 顶部添加：

```
/// 从请求上下文中安全提取已认证的用户 ID
String? _getAuthenticatedUserId(Request request) {
  final auth = request.context['auth'] as Map<String, dynamic>?;
  if (auth == null) return null;
  final uid = auth['uid'];
  return uid?.toString();
}
```

步骤 2：修复 `_saveUserLocationHandler`（约第 120 行）

将：

```
final userId = data['userId'];
```

改为：

```
final userId = _getAuthenticatedUserId(request);
if (userId == null) {
  return Response.unauthorized(
    json.encode({'error': '未认证'}),
    headers: {'Content-Type': 'application/json'},
  );
}
```

步骤 3：修复 `_updateUserProfile`（约第 227 行）

将：

```
if (!data.containsKey('userId') || data['userId'] == null) {
  return Response.badRequest(...);
}
```

改为：

```
final userId = _getAuthenticatedUserId(request);
if (userId == null) {
  return Response.unauthorized(
    json.encode({'error': '未认证'}),
    headers: {'Content-Type': 'application/json'},
  );
}
// 后续使用此 userId 而非 data['userId']
```

步骤 4：修复 `_getMyProfile`（约第 480 行）

将：

```
final userId = request.url.queryParameters['userId']
    ?? request.headers['X-User-Id'];
```

改为：

```
final userId = _getAuthenticatedUserId(request);
```

验收标准

- [ ] 所有 `/api/users/me/*` 接口仅使用 JWT 中的 uid
- [ ] 用伪造 userId 的请求返回 401 或操作自身数据
- [ ] 手动测试：用户 A 的 token 无法读取/修改用户 B 的资料

任务 1.4：前端令牌存储加密

对应问题：C-3

优先级：🔴 P0

涉及文件

`frontend/plugins/authclient/lib/src/services/tools/local_storage.dart`

实现步骤

步骤 1：添加依赖

在 `frontend/plugins/authclient/pubspec.yaml` 添加：

```
dependencies:
  flutter_secure_storage: ^9.0.0
```

步骤 2：重构`SharedPreferencesAuthStorage`

```
import 'package:flutter_secure_storage/flutter_secure_storage.dart';

class SecureAuthStorage implements IAuthStorage {
  static const _storage = FlutterSecureStorage(
    aOptions: AndroidOptions(encryptedSharedPreferences: true),
  );

  static const _kAuthToken = 'auth_token';
  static const _kRefreshToken = 'auth_refresh_token';
  static const _kUserJson = 'auth_user_json';
  // 注意：不再存储用户名和密码

  @override
  Future<void> saveSession({required Tokens tokens, Map<String, dynamic>?
user}) async {
    await _storage.write(key: _kAuthToken, value:
jsonEncode(tokens.toJson()));
    if (user != null) {
      await _storage.write(key: _kUserJson, value: jsonEncode(user));
    }
  }
```

```
  @override
  Future<void> clearSession() async {
    await _storage.deleteAll();
  }

  // ... 其他方法类似迁移
}
```

步骤 3：删除密码存储功能

移除 `saveCredentials` 中对密码的存储。如需"记住密码"功能，仅存储 refresh token。

验收标准

- [ ] `grep -r "SharedPreferences" frontend/plugins/authclient/` 仅出现在迁移代码中
- [ ] 令牌和用户信息通过 `flutter_secure_storage` 存储
- [ ] 不再存储明文密码
- [ ] Android: 验证数据在 `/data/data/<package>/shared_prefs/` 中不可读
- [ ] iOS: 验证数据存储在 Keychain 中

任务 1.5：WebSocket Token 传输方式改造

对应问题：C-4

优先级：🔴 P0

涉及文件

端	文件
前端	`frontend/plugins/chatclient/lib/data/network/websocket_manager.dart`
后端	后端 WebSocket 握手处理（chatcore 插件中）

实现步骤

步骤 1：前端 — 从 URL 参数移至 Header

在 `websocket_manager.dart` 的 `connect()` 方法中：

将：

```
String wsUrl = '$_baseUrl/$userId?name=${Uri.encodeComponent(_userName ??
'Anonymous')}';
if (token != null && token.isNotEmpty) {
  wsUrl += '&token=${Uri.encodeComponent(token)}';
}
_socket = await WebSocket.connect(wsUrl, headers: _headers);
```

改为：

```
String wsUrl = '$_baseUrl/$userId?name=${Uri.encodeComponent(_userName ??
'Anonymous')}';
// Token 通过 Header 传递，不放在 URL 中
final connectHeaders = Map<String, dynamic>.from(_headers);
if (token != null && token.isNotEmpty) {
  connectHeaders['Authorization'] = 'Bearer $token';
}
_socket = await WebSocket.connect(wsUrl, headers: connectHeaders);
```

步骤 2：后端 — 从 Header 读取 Token

在 chatcore 的 WebSocket 握手处理中，确保从 `Authorization` header 读取 token（而非 URL 参数）。

验收标准

- [ ] WebSocket 连接 URL 中不包含 token 参数
- [ ] 服务端日志中不出现 token 值
- [ ] WebSocket 连接认证正常

任务 1.6：启用 JWT 过期验证

对应问题：C-5

优先级：🔴 P0

涉及文件

`backend/plugins/authcore/lib/src/jwt_token.dart`

实现步骤

步骤 1：修改 `parseToken` 方法

将：

```
static Map<String, dynamic> parseToken(String token, String tokenSecret) {
  final jwt = JWT.tryVerify(token, SecretKey(tokenSecret),
      checkExpiresIn: false,
      checkHeaderType: false,
      checkNotBefore: false);
  if (jwt == null) return {};
  return jwt.payload;
}
```

改为：

```
static Map<String, dynamic> parseToken(String token, String tokenSecret) {
  final jwt = JWT.tryVerify(token, SecretKey(tokenSecret),
      checkExpiresIn: true,   // 启用过期验证
```

```
      checkHeaderType: false,
      checkNotBefore: true);  // 启用 nbf 验证
  if (jwt == null) return {};
  return jwt.payload;
}
```

步骤 2：统一令牌时间戳格式

检查令牌签发时 `exp` 字段的单位。JWT 标准使用 **Unix 秒级时间戳**（非毫秒）。确认 `JwtToken` 类中 `issueAccessToken` 方法的 `exp` 值符合标准：

```
// 正确：秒级
'exp': DateTime.now().add(duration).millisecondsSinceEpoch ~/ 1000,
```

步骤 3：清理 auth_middleware 中的冗余手动检查

启用库级别验证后，可移除 `auth_middleware.dart` 中的手动过期判断逻辑，减少重复。

验收标准

- [ ] 过期的 JWT 令牌返回 401
- [ ] `exp` 字段使用标准 Unix 秒级时间戳
- [ ] auth_middleware 无冗余过期检查
- [ ] 刷新令牌流程正常工作

任务 1.7：前端路由添加鉴权守卫

对应问题：C-6

优先级：🔴 P0

涉及文件

文件	修改
`frontend/lib/app/app_routes.dart`	为受保护路由添加 middleware
新建 `frontend/lib/app/core/middleware/auth_guard.dart`	创建守卫

实现步骤

步骤 1：创建 AuthGuardMiddleware

```
// frontend/lib/app/core/middleware/auth_guard.dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';
import 'package:authclient/authclient.dart';
```

```
class AuthGuardMiddleware extends GetMiddleware {
  @override
  int? get priority => 1;

  @override
  RouteSettings? redirect(String? route) {
    final authController = Get.find<AuthController>();
    if (authController.state.value != AuthState.authenticated) {
      return const RouteSettings(name: '/login');
    }
    return null;
  }
}
```

步骤 2：为受保护路由添加守卫

```
// app_routes.dart 中
GetPage(
  name: home,
  page: () => const HomeView(),
  binding: HomeBinding(),
  middlewares: [AuthGuardMiddleware()], // 添加
),
GetPage(
  name: settings,
  page: () => const SettingsView(),
  binding: SettingsBinding(),
  middlewares: [AuthGuardMiddleware()], // 添加
),
GetPage(
  name: deviceManagement,
  page: () => const DeviceManagementView(),
  binding: DeviceManagementBinding(),
  middlewares: [AuthGuardMiddleware()], // 添加
),
// ... 对 profileEdit、changePassword、userDetail 同样处理
```

不需要守卫的路由：`/`（splash）、`/login`、`/register`

验收标准

- [ ] 未登录时访问 `/home` 自动跳转 `/login`
- [ ] Deep Link `withyou://settings` 在未登录时跳转登录页
- [ ] 登录后正常跳转到目标页面

任务 1.8：全链路 HTTPS/WSS 改造

对应问题：H-10

优先级：🔴 P0

实现步骤

步骤 1：服务端部署 TLS 证书

- 为 `www.ycwithyou.com` 申请 TLS 证书（Let's Encrypt 或购买）
- 在 Nginx 反向代理前配置 HTTPS：

```
server {
    listen 443 ssl;
    server_name www.ycwithyou.com;

    ssl_certificate     /etc/ssl/certs/ycwithyou.pem;
    ssl_certificate_key /etc/ssl/private/ycwithyou.key;

    location / {
        proxy_pass http://127.0.0.1:8080;
    }

    location /ws/ {
        proxy_pass http://127.0.0.1:8080;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }
}

server {
    listen 80;
    server_name www.ycwithyou.com;
    return 301 https://$host$request_uri;
}
```

步骤 2：前端配置改为 HTTPS/WSS

`frontend/lib/configure.dart`：

```
static const remoteServerIp = 'www.ycwithyou.com';
static const remoteServerScheme = 'https';
static const wsScheme = 'wss';
```

`frontend/plugins/chatclient/` WebSocket URL：

```
// ws:// → wss://
final wsUrl = 'wss://$host:$port/ws/chat';
```

步骤 3：Android 禁用明文流量

`frontend/android/app/src/main/AndroidManifest.xml`：

```
<application
    android:usesCleartextTraffic="false"  <!-- 改为 false -->
```

验收标准

- [ ] `curl http://www.ycwithyou.com` 返回 301 重定向到 HTTPS

- [ ] 前端所有 API 请求使用 HTTPS
- [ ] WebSocket 连接使用 WSS
- [ ] Android `usesCleartextTraffic="false"`

任务 1.9：InMemoryTokenService 替换为持久化实现

对应问题：C-7

优先级：🔴 P0

涉及文件

文件	修改
`backend/bin/server.dart`	替换 InMemoryTokenService 和 InMemoryPasswordHasher
`backend/plugins/authcore/lib/`	可能需新增 MySQL 版 TokenService
`backend/pubspec.yaml`	添加 `bcrypt` 或 `dbcrypt` 依赖

实现步骤

步骤 1：实现 MySqlTokenService

```
class MySqlTokenService implements TokenService {
  @override
  Future<String> issueAccessToken(Map<String, dynamic> claims, Duration
expiry) async {
    // JWT 签发逻辑（无需持久化，JWT 自包含）
    final jwt = JWT(claims);
    return jwt.sign(SecretKey(Configure.jwtSecret),
        expiresIn: expiry);
  }

  @override
  Future<String> rotateRefreshToken(String? oldToken, int userId) async {
    // 在 auth_sessions 表中创建新的刷新令牌
    final newToken = _generateSecureToken();
    final hash = sha256.convert(utf8.encode(newToken)).toString();

    if (oldToken != null) {
      await _revokeToken(oldToken);
    }
    await DatabaseService.instance.insert(
      'INSERT INTO auth_sessions (user_id, token_hash, expires_at) VALUES
(?, ?, ?)',
      [userId, hash, DateTime.now().add(Duration(days: 30)).toUtc()],
    );
    return newToken;
  }
```

```
  @override
  Future<bool> validateRefreshToken(String token) async {
    final hash = sha256.convert(utf8.encode(token)).toString();
    final rs = await DatabaseService.instance.query(
      'SELECT id FROM auth_sessions WHERE token_hash = ? AND expires_at >
NOW() AND revoked_at IS NULL',
      [hash],
    );
    return rs.isNotEmpty;
  }
}
```

步骤 2：替换 InMemoryPasswordHasher 为 BCrypt

```
import 'package:dbcrypt/dbcrypt.dart';

class BcryptPasswordHasher implements PasswordHasher {
  final _bcrypt = DBCrypt();

  @override
  String hash(String password) {
    return _bcrypt.hashpw(password, _bcrypt.gensalt());
  }

  @override
  bool verify(String password, String hash) {
    return _bcrypt.checkpw(password, hash);
  }
}
```

步骤 3：修改 `server.dart`

```
// 替换
final tokenService = MySqlTokenService();
final hasher = BcryptPasswordHasher();
```

> ⚠️ 注意：替换 PasswordHasher 后，现有用户的密码哈希格式会不兼容。需要制定数据迁移策略（如：首次登录时用旧 hasher 验证 → 用新 hasher 重新哈希并存储）。

验收标准

- [ ] 服务重启后刷新令牌仍然有效
- [ ] 密码使用 bcrypt 哈希（`$2b$` 前缀）
- [ ] 旧用户可通过迁移逻辑正常登录

阶段二：安全加固（第 2-3 周）

任务 2.1：实现 API 速率限制中间件

对应问题：H-1

优先级：🟠 P1

涉及文件

新建 `backend/lib/middleware/rate_limit_middleware.dart`

实现步骤

步骤 1：实现滑动窗口限流中间件

```
import 'package:shelf/shelf.dart';
import 'dart:collection';

class RateLimiter {
  final int maxRequests;
  final Duration window;
  final Map<String, Queue<DateTime>> _requests = {};

  RateLimiter({required this.maxRequests, required this.window});

  bool isAllowed(String clientId) {
    final now = DateTime.now();
    _requests.putIfAbsent(clientId, () => Queue());
    final queue = _requests[clientId]!;

    // 清除窗口外的请求
    while (queue.isNotEmpty && now.difference(queue.first) > window) {
      queue.removeFirst();
    }

    if (queue.length >= maxRequests) return false;
    queue.add(now);
    return true;
  }
}

Middleware rateLimitMiddleware({
  int maxRequests = 60,
  Duration window = const Duration(minutes: 1),
}) {
  final limiter = RateLimiter(maxRequests: maxRequests, window: window);

  return (Handler handler) {
    return (Request request) {
      final clientIp = request.headers['X-Forwarded-For']
          ?? request.headers['X-Real-IP']
          ?? 'unknown';

      if (!limiter.isAllowed(clientIp)) {
        return Response(429,
          body: json.encode({'error': '请求过于频繁，请稍后重试'}),
```

```
          headers: {
            'Content-Type': 'application/json',
            'Retry-After': '60',
          },
        );
      }
      return handler(request);
    };
  };
}
```

步骤 2：登录接口特别限流

对 `/api/auth/login` 实施更严格的限制（如每分钟 5 次），并实现账户锁定：

```
// 连续 5 次登录失败后锁定 15 分钟
Middleware loginRateLimitMiddleware() {
  final limiter = RateLimiter(maxRequests: 5, window: Duration(minutes:
15));
  // ...
}
```

步骤 3：注册到中间件管线

在 `server.dart` 中：

```
final handler = Pipeline()
    .addMiddleware(logRequests())
    .addMiddleware(rateLimitMiddleware())        // 全局限流
    .addMiddleware(authMiddleware(tokenSecret))
    .addHandler(router);
```

验收标准

- [ ] 超过频率限制返回 HTTP 429
- [ ] 登录接口有独立的更严格限制
- [ ] 合法请求不受影响

任务 2.2：统一错误响应，隐藏内部信息

对应问题：H-2

优先级：🟠 P1

涉及文件

`backend/lib/routes/json_routes/api_routes.dart`

（所有 catch 块）

实现步骤

步骤 1：创建标准错误响应工具

```dart
// backend/lib/routes/error_response.dart
import 'dart:convert';
import 'dart:math';
import 'package:shelf/shelf.dart';

String _generateRequestId() =>
    DateTime.now().millisecondsSinceEpoch.toRadixString(36) +
    Random().nextInt(9999).toString().padLeft(4, '0');

Response errorResponse(int statusCode, String publicMessage, {Object?
internalError}) {
  final requestId = _generateRequestId();

  // 仅在服务端日志中记录详细错误
  if (internalError != null) {
    LoggerService.instance.severe(
      '[$requestId] $publicMessage: $internalError',
    );
  }

  return Response(statusCode,
    body: json.encode({
      'success': false,
      'error': publicMessage,
      'requestId': requestId,
    }),
    headers: {'Content-Type': 'application/json'},
  );
}
```

步骤 2：替换所有 catch 块中的错误暴露

将：

```dart
} catch (e, stackTrace) {
  return Response.internalServerError(
    body: json.encode({'error': '保存位置失败: ${e.toString()}'}),
  );
}
```

改为：

```dart
} catch (e, stackTrace) {
  return errorResponse(500, '操作失败，请稍后重试', internalError: e);
}
```

验收标准

- [ ] API 错误响应中不包含异常类名、SQL 语句、文件路径
- [ ] 每个错误带 `requestId` 便于排查
- [ ] 服务端日志有详细错误信息

任务 2.3：实现 SSL 证书固定

对应问题：H-3

优先级：🟠 P1

涉及文件

文件	修改
`frontend/plugins/chatclient/lib/data/network/websocket_manager.dart`	WebSocket 证书验证
前端 Dio HTTP 配置文件	HTTP 证书固定

实现步骤

步骤 1：HTTP 证书固定

```
// 在 Dio 配置中添加
(dio.httpClientAdapter as IOHttpClientAdapter).createHttpClient = () {
  final client = HttpClient();
  client.badCertificateCallback = (cert, host, port) {
    // 验证证书指纹
    final fingerprint = sha256.convert(cert.der).toString();
    return fingerprint == expectedFingerprint;
  };
  return client;
};
```

步骤 2：WebSocket 证书验证

```
final securityContext = SecurityContext.defaultContext;
final client = HttpClient(context: securityContext);
_socket = await WebSocket.connect(wsUrl,
  headers: connectHeaders,
  customClient: client,
);
```

验收标准

- [ ] 使用自签名证书的代理无法拦截通信
- [ ] 正常 TLS 证书正常工作
- [ ] 证书更换有明确的更新流程

任务 2.4：修复 WebSocket StreamSubscription 内存泄漏

对应问题：H-4

优先级：🟠 P1

涉及文件

`frontend/plugins/chatclient/lib/data/network/websocket_manager.dart`

实现步骤

步骤 1：保存 StreamSubscription 引用

在类中添加字段：

```
StreamSubscription? _socketSubscription;
```

步骤 2：修改连接建立逻辑

```
void _onConnectionEstablished() {
  // 先取消旧的订阅（如果有）
  _socketSubscription?.cancel();

  _socketSubscription = _socket!.listen(
    _onMessageReceived,
    onDone: () => _onConnectionClosed(/* ... */),
    onError: (error) => _onConnectionError(error),
  );
}
```

步骤 3：修改 dispose/disconnect 逻辑

```
Future<void> dispose() async {
  await _socketSubscription?.cancel();
  _socketSubscription = null;
  await disconnect();
}
```

验收标准

- [ ] 多次重连后内存不持续增长
- [ ] `dispose()` 后无残留监听器
- [ ] 使用 DevTools Memory 工具验证无泄漏

任务 2.5：后端输入校验加固

对应问题：H-5

优先级：🟠 P1

涉及文件

`backend/lib/routes/json_routes/api_routes.dart`

实现步骤

步骤 1：创建校验工具函数

```
// backend/lib/routes/validators.dart
```

```
/// 校验经纬度
(double, double)? validateCoordinates(dynamic lat, dynamic lon) {
  final latitude = double.tryParse(lat?.toString() ?? '');
  final longitude = double.tryParse(lon?.toString() ?? '');
  if (latitude == null || longitude == null) return null;
  if (latitude < -90 || latitude > 90) return null;
  if (longitude < -180 || longitude > 180) return null;
  return (latitude, longitude);
}

/// 校验字符串长度
String? validateString(dynamic value, {int maxLength = 500, bool required =
false}) {
  if (value == null) return required ? null : '';
  final s = value.toString();
  if (s.length > maxLength) return null;
  return s;
}
```

步骤 2：在`_saveUserLocationHandler`中使用

```
final coords = validateCoordinates(data['latitude'], data['longitude']);
if (coords == null) {
  return errorResponse(400, '无效的坐标值');
}
final (latitude, longitude) = coords;
```

步骤 3：在`_updateUserProfile`中添加字段长度校验

```
final displayName = validateString(data['displayName'], maxLength: 50);
final bio = validateString(data['bio'], maxLength: 500);
// ...
```

验收标准

- [ ] 非法坐标（如 `latitude: "abc"` ）返回 400
- [ ] 超长字段名被拒绝
- [ ] 正常请求不受影响

任务 2.6：清除所有 print 语句

对应问题：H-6

优先级：🟠 P1

实现步骤

步骤 1：后端 — 全局搜索并替换

```
# 查找所有 print 语句
```

```
grep -rn "print(" backend/bin/ backend/lib/ --include="*.dart"
```

将所有 `print('...')` 替换为 `LoggerService.instance.info('...')` 或对应级别。

步骤 2：前端 — 用 kDebugMode 包裹

```
grep -rn "debugPrint\|print(" frontend/lib/ frontend/plugins/ --
include="*.dart"
```

```
// 替换敏感信息的 print
if (kDebugMode) {
  debugPrint('连接状态: $state');
}
```

移除任何打印 token、密码、完整消息内容的语句。

步骤 3：添加 lint 规则

在 `analysis_options.yaml` 中：

```
linter:
  rules:
    - avoid_print  # 禁止在生产代码中使用 print
```

验收标准

- [ ] `grep -rn "print(" backend/lib/ backend/bin/ --include="*.dart"` 无结果
- [ ] 前端 `debugPrint` 仅在 `kDebugMode` 内使用且不含敏感数据
- [ ] analysis*options.yaml* 启用 `*avoid*print` 规则

任务 2.7：服务端组件故障隔离

对应问题：H-7

优先级：🟠 P1

涉及文件

`backend/bin/server.dart`

实现步骤

步骤 1：将每个组件的初始化包裹在独立 try-catch 中

```
// 核心组件 — 失败则终止
bool dbReady = false;
try {
  await _createTables();
  dbReady = true;
} catch (e) {
```

```
  log.severe('数据库初始化失败，服务终止：$e');
  exit(1);
}

// 可降级组件 — 失败则记录日志并继续
bool chatReady = false;
try {
  await _initChatCore();
  chatReady = true;
} catch (e) {
  log.severe('ChatCore 初始化失败（聊天功能不可用）：$e');
}

bool pushReady = false;
try {
  await _initPushService();
  pushReady = true;
} catch (e) {
  log.severe('推送服务初始化失败（推送功能不可用）：$e');
}
```

步骤 2：健康检查接口细化

```
// GET /health 返回各组件状态
return Response.ok(json.encode({
  'status': 'running',
  'components': {
    'database': dbReady,
    'chat': chatReady,
    'push': pushReady,
  },
  'version': Configure.version,
  'uptime': _uptime(),
}));
```

验收标准

- [ ] 推送服务初始化失败时，认证和聊天仍可用
- [ ] `/health` 接口返回各组件状态
- [ ] 数据库不可用时服务拒绝启动

任务 2.8：修复离线消息队列发送逻辑

对应问题：H-8

优先级：🟠 P1

涉及文件

`frontend/plugins/chatclient/lib/data/network/websocket_manager.dart`

实现步骤

```
Future<void> _flushMessageQueue() async {
  final messages = List.from(_pendingMessages);
  _pendingMessages.clear();

  for (final message in messages) {
    sendMessage(message);
    await Future.delayed(const Duration(milliseconds: 50)); // await 延迟
  }
}
```

验收标准

- [ ] 离线消息按间隔逐条发送
- [ ] 服务端不因瞬间大量消息而过载

任务 2.9：退出登录时清除所有本地凭证

对应问题：H-9

优先级：🟠 P1

涉及文件

`frontend/plugins/authclient/lib/src/` — AuthController 的 logout 方法

实现步骤

```
Future<void> logout() async {
  // 1. 通知服务端注销会话
  try {
    await _apiService.logout();
  } catch (_) {}

  // 2. 清除所有本地存储
  await _storage.clearSession();  // 令牌
  await _storage.clearCredentials(); // 用户名（如有）

  // 3. 断开 WebSocket
  WebSocketManager.instance.disconnect();

  // 4. 清除本地数据库缓存
  await DatabaseManager.instance.clearAll();

  // 5. 更新状态
  state.value = AuthState.unauthenticated;
  user.value = null;
}
```

验收标准

- [ ] 登出后 `flutter_secure_storage` 中无残留数据
- [ ] 登出后本地 SQLite 聊天记录已清除
- [ ] 重新打开 App 时进入登录页

任务 2.10：推送 debugMode 关闭

对应问题：C-1 相关

优先级：🟠 P1

涉及文件

`frontend/lib/app/core/services/push/push_handler.dart`

实现步骤

```
// 将
debugMode: true,
// 改为
debugMode: kDebugMode,
```

验收标准

- [ ] Release 包不输出推送调试日志
- [ ] Debug 模式下仍可看到日志

阶段三：工程规范化（第 4-8 周）

任务 3.1：数据库层异常处理规范化

对应问题：M-1

优先级：🟡 P2

涉及文件

`backend/plugins/authsql/lib/src/sql_adapters.dart`

实现步骤

将所有 `catch (_) { return null; }` 改为：

```
} on MySqlException catch (e) {
  LoggerService.instance.warning('数据库查询失败: $e');
  return null;  // 或根据情况抛出自定义异常
} catch (e) {
```

```
  LoggerService.instance.severe('未预期的错误: $e');
  rethrow;  // 未知异常应向上传播
}
```

验收标准

- [ ] 无静默 `catch (_)`
- [ ] 数据库错误有日志记录
- [ ] 区分"未找到"（返回 null）和"查询出错"（抛异常）

任务 3.2：DTO 版本管理规范化

对应问题：M-2

优先级：🟡 P2

实现步骤

1. 为 `chat_dtos` 仓库创建语义版本 tag
2. `pubspec.yaml` 中锁定版本：

```yaml
chat_dtos:
  git:
    url: https://github.com/xxx/chat_dtos.git
    ref: v1.0.0  # 锁定到具体 tag
```

1. Breaking change 时 bump major version，添加 CHANGELOG

验收标准

- [ ] chat_dtos 有版本 tag
- [ ] 前后端 pubspec.yaml 引用具体 tag
- [ ] 有 CHANGELOG.md

任务 3.3：统一 API 错误码体系

对应问题：M-3

优先级：🟡 P2

实现步骤

步骤 1：定义错误码枚举

```
// backend/lib/routes/error_codes.dart
enum ApiError {
```

```
  unauthorized(40100, '未认证'),
  tokenExpired(40101, '令牌已过期'),
  tokenInvalid(40102, '令牌无效'),
  forbidden(40300, '无权限'),
  notFound(40400, '资源不存在'),
  rateLimited(42900, '请求过于频繁'),
  validationError(42200, '请求参数错误'),
  internalError(50000, '服务器内部错误');

  final int code;
  final String message;
  const ApiError(this.code, this.message);
}
```

步骤 2：统一响应格式

```
// 成功
{ "code": 0, "data": { ... }, "requestId": "abc123" }

// 失败
{ "code": 40101, "message": "令牌已过期", "requestId": "abc123" }
```

步骤 3：逐步迁移所有路由使用统一格式

验收标准

- [ ] 所有 API 响应格式统一
- [ ] 前端可按 code 字段做差异化处理
- [ ] 有错误码文档

任务 3.4：引入数据库迁移管理

对应问题：M-4

优先级：🟡 P2

实现步骤

步骤 1：创建迁移框架

```
// backend/lib/migration/migrator.dart

class Migration {
  final int version;
  final String description;
  final String upSql;

  const Migration({required this.version, required this.description,
required this.upSql});
}
```

```
class Migrator {
  static Future<void> migrate(List<Migration> migrations) async {
    // 1. 创建 schema_migrations 表
    await DatabaseService.instance.query('''
      CREATE TABLE IF NOT EXISTS schema_migrations (
        version INT PRIMARY KEY,
        description VARCHAR(255),
        applied_at DATETIME DEFAULT CURRENT_TIMESTAMP
      )
    ''');

    // 2. 获取当前版本
    final rs = await DatabaseService.instance.query(
      'SELECT MAX(version) as v FROM schema_migrations'
    );
    final currentVersion = rs.first['v'] as int? ?? 0;

    // 3. 执行未应用的迁移
    for (final m in migrations.where((m) => m.version > currentVersion)) {
      await DatabaseService.instance.transaction((txn) async {
        await txn.query(m.upSql);
        await txn.query(
          'INSERT INTO schema_migrations (version, description) VALUES
(?, ?)',
          [m.version, m.description],
        );
      });
      log.info('迁移 v${m.version} 完成: ${m.description}');
    }
  }
}
```

步骤 2：将现有 CREATE TABLE 转为初始迁移

```
const migrations = [
  Migration(
    version: 1,
    description: '初始表结构',
    upSql: '''
      CREATE TABLE IF NOT EXISTS auth_users (...);
      CREATE TABLE IF NOT EXISTS user_profiles (...);
      ...
    ''',
  ),
  // 后续字段变更在此添加
];
```

验收标准

- [ ] `schema_migrations` 表记录所有已应用的迁移
- [ ] 新字段通过添加 Migration 而非手动 ALTER 实现
- [ ] 服务启动时自动执行未应用的迁移

任务 3.5：chatcore 纳入 Monorepo 管理

对应问题：M-6

优先级：🟡 P2

实现步骤

方案 A：Git Submodule

```
cd backend/plugins
git submodule add https://github.com/xxx/chatcore.git chatcore
```

方案 B：直接复制并合入 Monorepo（推荐）

将 chatcore 代码直接放入 `backend/plugins/chatcore/` ，统一管理。

验收标准

- [ ] chatcore 代码在本地可直接修改和调试
- [ ] 不需要跨仓库提交来修改 IM 核心逻辑

任务 3.6：推送服务重试与降级机制

对应问题：M-7

优先级：🟡 P2

涉及文件

`backend/plugins/wxtpushserver/lib/`

实现步骤

```
Future<PushResult> sendWithRetry(PushTarget target, PushMessage message,
{int maxRetries = 3}) async {
  int attempt = 0;
  Duration delay = const Duration(seconds: 1);

  while (attempt < maxRetries) {
    try {
      return await sendToToken(target, message);
    } catch (e) {
      attempt++;
      if (attempt >= maxRetries) {
        log.severe('推送最终失败 [${target.vendor}]: $e');
        // 写入重试队列或死信队列
        await _enqueueForRetry(target, message);
        return PushResult(success: false, error: e.toString());
      }
      log.warning('推送重试 $attempt/$maxRetries [${target.vendor}]');
```

```
      await Future.delayed(delay);
      delay *= 2; // 指数退避
    }
  }
  return PushResult(success: false, error: '超过最大重试次数');
}
```

验收标准

- [ ] 推送失败自动重试最多 3 次
- [ ] 重试使用指数退避
- [ ] 最终失败记录日志

任务 3.7：前端 SQLite 数据库加密

对应问题：M-8

优先级：🟡 P2

实现步骤

1. 替换 `sqflite` 为 `sqflite_sqlcipher`
2. 在打开数据库时提供加密密钥：

```dart
final db = await openDatabase(
path,
password: await getOrCreateDbKey(), // 从 flutter secure_storage 获取
);
```

1. 首次安装时生成随机密钥存储在 `flutter_secure_storage`
2. 提供从未加密到加密的迁移逻辑

验收标准

- [ ] 数据库文件无法用 sqlite3 直接打开
- [ ] 应用功能正常
- [ ] 旧版本升级后数据迁移成功

任务 3.8：引入 API 版本管理

对应问题：L-1

优先级：🟡 P2

实现步骤

1. 所有新路由使用 `/api/v1/` 前缀
2. 旧路由 `/api/users/` 映射到 `/api/v1/users/`（兼容层）
3. 客户端请求头添加 `X-API-Version: 1`
4. 未来 breaking change 创建 `/api/v2/` 并共存

```
// router 挂载
router.mount('/api/v1/', apiV1Router);
router.mount('/api/', apiV1Router); // 兼容旧客户端
```

验收标准

- [ ] `/api/v1/users/me/profile` 可用
- [ ] `/api/users/me/profile` 通过兼容层仍可用
- [ ] 前端新版本使用 v1 前缀

阶段四：架构演进（第 9-16 周）

任务 4.1：状态持久化支持水平扩展

对应问题：M-5

优先级：🔵 P3

实现步骤

1. 将 `ConnectionService` 的在线用户状态存储到 Redis
2. WebSocket 消息广播通过 Redis Pub/Sub 实现跨节点分发
3. Token 验证查询 MySQL（已在任务 1.9 完成基础）
4. 使用 Nginx upstream + sticky session 作为过渡方案

验收标准

- [ ] 两个后端实例同时运行时，用户 A 连节点 1，用户 B 连节点 2，可互发消息
- [ ] 单节点重启不影响另一节点上的连接

任务 4.2：可观测性体系建设

对应问题：L-4

优先级：🔵 P3

实现步骤

1. 后端接入结构化 JSON 日志（替代文本日志）
2. 引入关键指标：
    - HTTP 请求延迟 (p50/p95/p99)
    - WebSocket 活跃连接数
    - 消息发送/接收 QPS
    - 推送成功/失败率
    - 数据库连接池利用率
1. 搭建 Grafana 仪表盘
2. 设置告警规则（错误率 > 5%、连接池耗尽等）

验收标准

- [ ] Grafana 仪表盘可用
- [ ] 关键指标有告警
- [ ] 问题可通过 requestId 追踪完整链路

任务 4.3：前端架构分层重构

对应问题：L-5

优先级：🔵 P3

实现步骤

引入 UseCase 层：

```
View → Controller → UseCase → Repository → DataSource
                                  ↑
                              (业务规则)
```

示例:

```dart
// use_cases/send_message_use_case.dart
class SendMessageUseCase {
  final MessageRepository _repo;
  final WebSocketManager _ws;

  Future<void> execute(String roomId, String content) async {
    // 1. 校验消息
    if (content.trim().isEmpty) throw EmptyMessageException();

    // 2. 创建本地消息记录
    final message = await _repo.createLocal(roomId, content);

    // 3. 通过 WebSocket 发送
    await _ws.sendMessage(message.toJson());
```

```
    // 4. 标记发送状态
    await _repo.updateStatus(message.id, MessageStatus.sent);
  }
}
```

验收标准

- [ ] Controller 不直接调用 API
- [ ] 业务逻辑可脱离 UI 独立测试
- [ ] UseCase 有单元测试

任务 4.4：建立测试体系与 CI 流水线

对应问题：L-6

优先级：🔵 P3

实现步骤

步骤 1：后端单元测试

```
backend/test/
├── middleware/
│   └── auth_middleware_test.dart
├── routes/
│   └── api_routes_test.dart
├── plugins/
│   ├── authcore_test.dart
│   └── chatcore_test.dart
└── integration/
    └── auth_flow_test.dart
```

步骤 2：前端单元测试

```
frontend/test/
├── controllers/
│   └── auth_controller_test.dart
├── use_cases/
│   └── send_message_test.dart
└── widgets/
    └── chat_input_test.dart
```

步骤 3：CI 流水线（GitHub Actions）

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  backend-test:
    runs-on: ubuntu-latest
    steps:
      - uses: dart-lang/setup-dart@v1
```

```
      - run: cd backend && dart pub get && dart test

  frontend-test:
    runs-on: ubuntu-latest
    steps:
      - uses: subosito/flutter-action@v2
      - run: cd frontend && flutter pub get && flutter test

  lint:
    runs-on: ubuntu-latest
    steps:
      - run: cd backend && dart analyze
      - run: cd frontend && flutter analyze
```

验收标准

- [ ] 后端核心逻辑测试覆盖率 > 60%
- [ ] 前端核心逻辑测试覆盖率 > 60%
- [ ] PR 合入需通过 CI 检查
- [ ] `dart analyze` / `flutter analyze` 无错误

附录

A. 任务依赖关系

```
1.1 (轮换凭证)
 └→ 1.2 (移除硬编码) ──→ 1.8 (HTTPS)

1.3 (修复 userId) ── 独立
1.4 (安全存储) ── 独立
1.5 (WS Token) ── 依赖 1.8
1.6 (JWT 过期) ── 独立
1.7 (路由守卫) ── 独立
1.9 (持久化 Token) ── 独立

2.1 (速率限制) ── 独立
2.2 (错误响应) ── 独立
2.3 (证书固定) ── 依赖 1.8
2.4 (内存泄漏) ── 独立
2.5 (输入校验) ── 独立
2.6 (清除 print) ── 独立
2.7 (故障隔离) ── 独立
2.8 (消息队列) ── 独立
2.9 (登出清除) ── 依赖 1.4
2.10 (debugMode) ── 独立

3.x ── 各自独立，可并行
```

```
4.x ── 依赖阶段 1-3 完成
```

B. 风险评估

任务	风险	缓解措施
1.1 轮换凭证	服务中断	先部署新凭证再废旧的，错峰操作
1.2 移除硬编码	前端需发版	先发后端，再发前端
1.8 HTTPS	证书配置错误导致不可用	先在测试环境验证，保留 HTTP 回退
1.9 替换 PasswordHasher	现有用户无法登录	实现双 hasher 过渡策略
3.4 数据库迁移	生产表被修改	先备份，迁移脚本在测试环境验证
3.7 SQLite 加密	旧用户数据丢失	实现未加密 → 加密的迁移逻辑

C. 进度跟踪模板

阶段	任务	负责人	状态	完成日期
一	1.1 轮换凭证		□ 待开始	
一	1.2 移除硬编码		□ 待开始	
一	1.3 修复 userId		□ 待开始	
一	1.4 安全存储		□ 待开始	
一	1.5 WS Token		□ 待开始	
一	1.6 JWT 过期		□ 待开始	
一	1.7 路由守卫		□ 待开始	
一	1.8 HTTPS		□ 待开始	
一	1.9 持久化 Token		□ 待开始	
二	2.1 速率限制		□ 待开始	
二	2.2 错误响应		□ 待开始	
二	2.3 证书固定		□ 待开始	
二	2.4 内存泄漏		□ 待开始	
二	2.5 输入校验		□ 待开始	
二	2.6 清除 print		□ 待开始	
二	2.7 故障隔离		□ 待开始	
二	2.8 消息队列修复		□ 待开始	
二	2.9 登出清除		□ 待开始	
二	2.10 debugMode		□ 待开始	
三	3.1 异常处理		□ 待开始	
三	3.2 DTO 版本管理		□ 待开始	
三	3.3 错误码体系		□ 待开始	
三	3.4 数据库迁移		□ 待开始	
三	3.5 chatcore 管理		□ 待开始	
三	3.6 推送重试		□ 待开始	
三	3.7 SQLite 加密		□ 待开始	
三	3.8 API 版本管理		□ 待开始	
四	4.1 水平扩展		□ 待开始	
四	4.2 可观测性		□ 待开始	
四	4.3 前端分层		□ 待开始	
四	4.4 测试与 CI		□ 待开始	